SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	K R Gayana	**Reg. No.:**	192471039
Date:	18.08.2026	**Duration:**	60 minutes

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – CI/CD Pipeline Design Exercise

PROBLEM STATEMENT:

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

- **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
- **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
- Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
- Define appropriate **automated testing strategies** to be integrated into the pipeline.
- Design the **deployment strategy**, including development, testing/staging, and production environments.
- Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
- Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

1. Introduction and Project Context

Warehouses are rapidly transitioning from manual, paper-driven operations to intelligent, technology-enabled facilities. The assigned problem statement calls for an Intelligent AI-Based Warehouse Automation and Inventory Tracking Platform that combines RFID tagging, IoT sensor networks, autonomous mobile robots (AMRs), AI-driven storage optimization, and cloud computing to deliver real-time inventory visibility, faster order fulfillment, and reduced operational costs.

Manual warehouse operations are prone to human error during stock counts, slow to reconcile discrepancies, and offer little live visibility into where inventory physically sits at any given moment. This drives the need for a platform that continuously senses inventory state through RFID/IoT hardware, applies AI to recommend optimal storage and retrieval decisions, and exposes that intelligence through real-time dashboards and reports to warehouse managers, logistics coordinators, and customers.

Because the platform integrates several fast-moving components — an AI optimization engine, an RFID/IoT ingestion layer, a real-time tracking dashboard, and warehouse operations APIs — it must be built and released using a disciplined, automated CI/CD pipeline. This document analyzes the CI/CD requirements of the platform and presents the full pipeline architecture, tool selection, testing strategy, deployment strategy, and operational safeguards (security, quality gates, notifications, and rollback) needed to move code safely and quickly from commit to production.

1.1 Scope of This Document

This document covers the design of the CI/CD pipeline only — i.e., how code, configuration, and ML model changes for the warehouse platform move from a developer's commit through automated build, test, and quality checks, into Development, Staging, and Production environments, with monitoring and rollback once live. It does not cover the detailed functional/system design of the warehouse platform itself, which would be addressed in a separate Software Requirements Specification (SRS) and System Design document.

1.2 Key Stakeholders

- Development Teams — backend, front-end, AI/ML, and IoT firmware engineers who commit code and consume pipeline feedback.
- QA / Test Engineers — design and maintain automated test suites, and perform UAT sign-off in Staging.
- DevOps / Platform Engineers — own and maintain the CI/CD pipeline, infrastructure-as-code, and Kubernetes clusters.
- Release Manager / Product Owner — approves promotion from Staging to Production.
- Warehouse Operations Team — end users of the platform; provide UAT feedback and are directly affected by deployment quality and downtime.
- Security / Compliance Team — reviews vulnerability scan results and enforces data-protection requirements for IoT/RFID data.

1.3 Objectives-to-Pipeline Mapping

The table below maps each objective from the assigned problem statement to the specific CI/CD pipeline capability that supports it, showing why each stage of the pipeline exists.

Problem Statement Objective	Supporting CI/CD Pipeline Capability
Automate warehouse inventory tracking	Automated build/test/deploy of the RFID/IoT ingestion service on every code change, ensuring tracking logic is always validated before release.
Optimize storage allocation using AI	Dedicated MLOps sub-pipeline (model training, validation, versioning via MLflow) integrated into the same promotion path.
Monitor inventory movement in real time	Continuous deployment of the streaming ingestion layer (Kafka/MQTT) plus Prometheus/Grafana monitoring in Production.
Improve order fulfillment efficiency	Performance/load testing stage validates order-processing latency before every production release.
Reduce inventory discrepancies	Layered automated testing (unit, integration, regression) catches logic errors before they reach live inventory data.
Generate warehouse performance reports	Automated deployment of the reporting/dashboard module with regression tests protecting report accuracy.
Integrate RFID-based inventory management	Dedicated build/test pipeline for the RFID ingestion microservice, isolated from other services for independent releases.
Enhance warehouse productivity	Fast, reliable CI/CD reduces release friction, letting productivity-improving features reach warehouse staff sooner.

2. Analysis of Project Requirements and CI/CD Needs

The platform is composed of multiple interacting subsystems, each with distinct build, test, and release characteristics:

- RFID/IoT Ingestion Service — high-frequency event ingestion from RFID readers and IoT sensors (edge/gateway software + message broker consumers).

- AI Storage Optimization Engine — machine learning models that recommend optimal storage/slotting locations and demand forecasts.
- Inventory & Order Management Core — transactional services handling stock updates, order fulfillment, and discrepancy reconciliation.
- Real-Time Monitoring Dashboard — web front-end providing live inventory visibility and warehouse performance reports.
- AMR/Robotics Integration Layer — APIs coordinating autonomous mobile robots for pick, pack, and putaway tasks.

From this architecture, the following CI/CD needs are identified:

- Frequent, automated integration of code changes from multiple teams (backend, AI/ML, IoT firmware, front-end) without breaking the shared platform.
- Automated builds that produce consistent, versioned, containerized artifacts for each microservice.
- Layered automated testing (unit, integration, performance, security) to catch inventory-accuracy defects before they reach a live warehouse floor.
- Static code analysis and security/dependency scanning, since the platform touches IoT devices and handles operational data that must be protected.
- Safe, staged deployment across Dev → Staging/UAT → Production environments, with the ability to test against simulated RFID/IoT data before going live.
- Real-time monitoring, alerting, and automated rollback, because warehouse operations are business-critical and downtime directly impacts order fulfillment.
- Traceability and reporting to support the objective of data-driven warehouse management and continuous improvement of the pipeline itself.

3. CI/CD Pipeline Architecture and Diagram

The pipeline below shows the full workflow from developer commit through to production deployment, monitoring, and feedback. It is organized into four logical bands: Source Management, Continuous Integration (Build → Test → Quality/Security → Package), Continuous Deployment (Dev → Staging → Approval → Production), and Operations (Monitoring, Alerts, Rollback, Feedback).

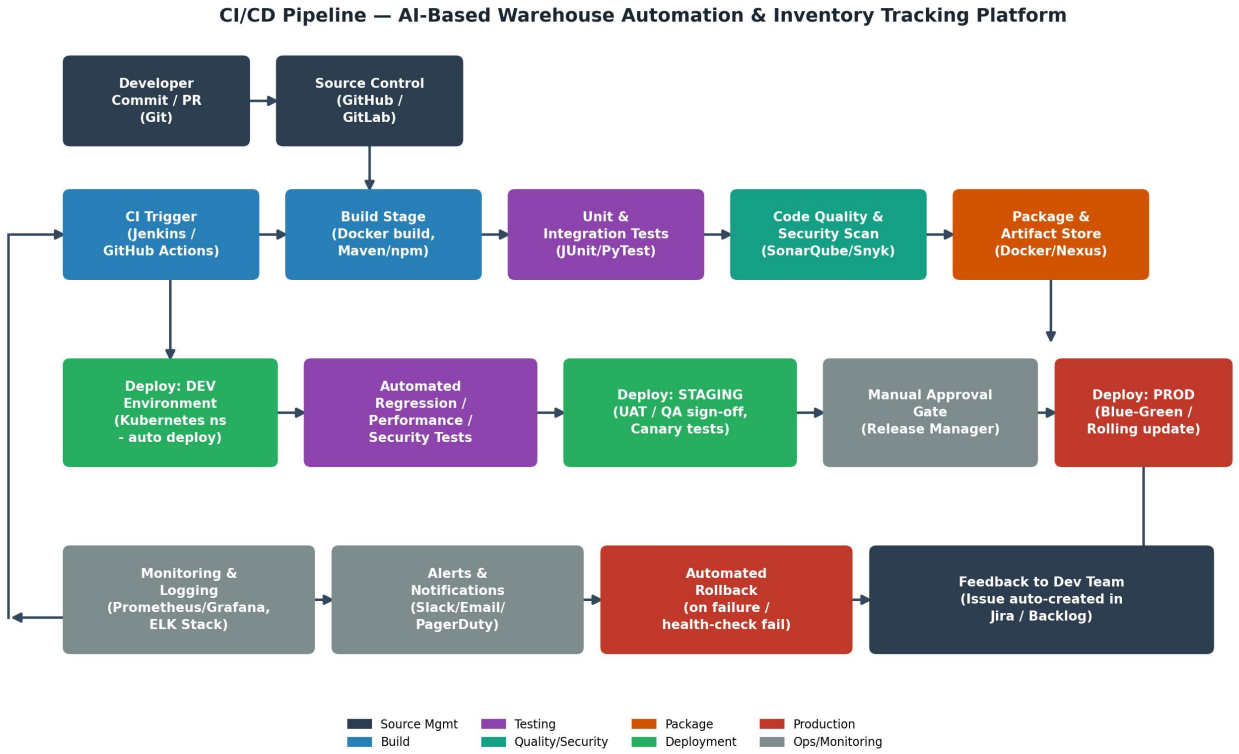

Figure 1: End-to-end CI/CD pipeline for the Warehouse Automation and Inventory Tracking Platform.

4. Pipeline Stages

4.1 Source Code Management (SCM)

All microservices (ingestion service, AI engine, inventory core, dashboard, AMR integration layer) are maintained in Git repositories using a trunk-based / feature-branch workflow. A commit or pull request automatically triggers the pipeline via a webhook. Branch protection rules require passing CI checks and at least one peer code review before merge to the main branch.

4.2 Build Stage

Each service is compiled/packaged and containerized independently. Dependency resolution (npm, pip/conda for ML components, Maven/Gradle for Java services) is cached to speed up builds. Build artifacts are tagged with the Git commit hash and semantic version for traceability.

4.3 Automated Testing Stage

Unit tests validate individual functions (e.g., RFID tag parsing, storage-slot scoring algorithm, order-fulfillment logic). Integration tests validate interactions such as RFID event ingestion updating inventory records in real time. This stage also runs contract tests between the AI optimization engine and the inventory core API.

4.4 Code Quality and Security Analysis

Static analysis (linting, complexity, code smells) and Software Composition Analysis (dependency vulnerability scanning) run automatically. A configurable quality gate blocks promotion if code coverage, maintainability, or critical vulnerability thresholds are not met.

4.5 Packaging and Artifact Management

Passing builds are packaged as versioned Docker images and pushed to a private container registry / artifact repository, along with the ML model artifacts for the AI storage-optimization engine (model registry).

4.6 Deployment Stage

Artifacts are progressively promoted through Development, Staging/UAT, and Production environments (see Section 6), with automated smoke tests, manual approval gating, and blue-green/rolling deployment to Production.

4.7 Monitoring, Alerting, and Feedback

Post-deployment, real-time monitoring and log aggregation track system health, RFID read accuracy, and order-processing latency. Alerts route to the operations/dev team, and failures trigger automated rollback plus an auto-created issue for the responsible team.

5. Tools and Technologies (with Justification)

Stage	Recommended Tool(s)	Justification
Source Control	Git + GitHub / GitLab	Industry-standard distributed VCS; built-in PR review workflow, branch protection, and native webhook triggers for CI.
CI Orchestration	Jenkins or GitHub Actions / GitLab CI	Mature, widely supported pipeline-as-code orchestration with rich plugin ecosystem for build, test, and deployment automation.
Build & Containerization	Docker, Maven/Gradle, npm, Conda	Docker ensures each microservice (ingestion, AI engine, dashboard) is built and shipped consistently across environments.
Unit / Integration Testing	JUnit, PyTest, Postman/Newman, Selenium	Covers backend logic, Python-based AI/ML components, API contract tests, and dashboard UI regression tests.
Code Quality	SonarQube	Automated static analysis, code

Stage	Recommended Tool(s)	Justification
		smell/complexity detection, and enforceable quality gates before merge/promotion.
Security Scanning	Snyk / OWASP Dependency-Check, Trivy	Scans dependencies and container images for known vulnerabilities — important given IoT/RFID data-handling exposure.
Artifact / Container Registry	Nexus / JFrog Artifactory, Docker Hub (private) / Harbor	Central, versioned storage for build artifacts, Docker images, and ML model files with role-based access control.
Model Registry / ML Ops	MLflow	Tracks and versions the AI storage-optimization models so deployments are reproducible and auditable.
Configuration/Deployment	Kubernetes + Helm, Terraform	Kubernetes provides scalable orchestration for microservices across Dev/Staging/Prod; Terraform manages infrastructure as code.
Message Broker (IoT/RFID)	Apache Kafka / MQTT	Handles high-throughput, low-latency streaming of RFID/IoT sensor events into the inventory pipeline.
Monitoring & Logging	Prometheus + Grafana, ELK Stack (Elasticsearch, Logstash, Kibana)	Real-time metrics dashboards and centralized log analysis to support the platform's real-time visibility objective.
Notifications	Slack / Microsoft Teams, Email, PagerDuty	Immediate pipeline and incident notifications to developers and on-call operations staff.
Cloud Platform	AWS / Azure / GCP (managed Kubernetes, object storage)	Elastic, on-demand compute/storage to scale with warehouse IoT data volume and support high-availability deployments.

6. Automated Testing Strategy

A layered testing strategy is embedded directly into the pipeline so that defects are caught as early as possible, reflecting the objective of reducing inventory discrepancies:

- Unit Testing: Validates isolated logic such as RFID tag decoding, discrepancy-detection rules, and storage-slot scoring formulas. Runs on every commit; target ≥ 80% code coverage.
- Integration Testing: Verifies that services work together correctly, e.g., an RFID read event correctly updates the inventory database and triggers a dashboard refresh.
- API / Contract Testing: Ensures the AI optimization engine, inventory core, and AMR integration layer honor agreed API contracts, preventing breaking changes across teams.
- Performance / Load Testing: Simulates peak RFID/IoT event throughput and concurrent order-fulfillment requests (e.g., using JMeter/Gatling) to validate the system meets latency SLAs.
- Security Testing: Automated SAST/DAST and dependency scanning at each build; periodic penetration testing before major releases.
- Regression Testing: Full automated regression suite runs before promotion to Staging, ensuring existing warehouse workflows (picking, packing, replenishment) are not broken.
- User Acceptance Testing (UAT): Conducted in the Staging environment with warehouse operations stakeholders before production sign-off.
- Canary / Smoke Testing: Lightweight post-deployment checks executed immediately after each environment promotion, including after Production rollout.

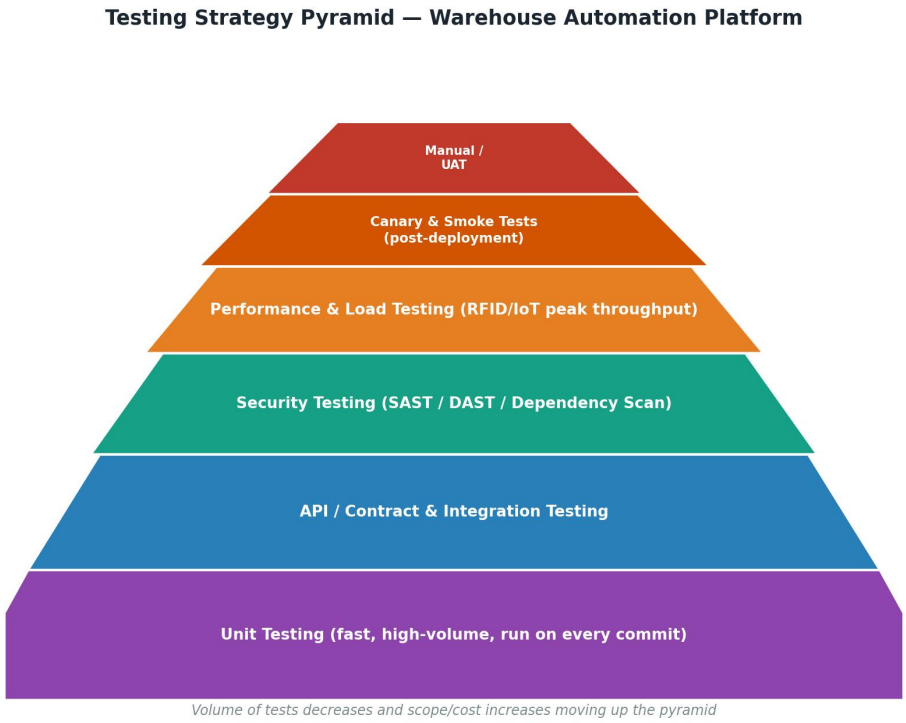

Figure 2: Testing strategy pyramid — high-volume fast checks at the base, narrower high-value checks near the top.

7. Deployment Strategy

7.1 Development Environment

Every successful build from the main/feature-integration branch is automatically deployed to a shared Dev environment (Kubernetes namespace) for continuous integration testing and early feedback. Data is simulated/synthetic RFID and IoT input.

7.2 Testing / Staging (UAT) Environment

Builds that pass automated regression, performance, and security testing are promoted to a Staging environment that mirrors production configuration. This environment is used for User Acceptance Testing with warehouse operations stakeholders and for canary validation against a subset of real or replayed RFID/IoT traffic.

7.3 Production Environment

Promotion to Production requires an explicit manual approval gate (release manager / product owner sign-off) after successful Staging validation. Deployment uses a blue-green or rolling-update strategy in Kubernetes so that the previous stable version remains available and traffic is only shifted to the new version once health checks pass, minimizing disruption to live warehouse operations.

7.4 Release Cadence

Core services follow a continuous delivery model (frequent, small, low-risk releases), while the AI storage-optimization model follows a separate, slower MLOps release cycle with additional validation (e.g., offline evaluation against historical warehouse data) before being promoted through the same Dev → Staging → Production path.

7.5 Environment Comparison

Aspect	Development	Staging / UAT	Production
Purpose	Continuous integration feedback for developers	Pre-release validation, UAT sign-off, canary testing	Live warehouse operations
Data	Synthetic / simulated RFID & IoT events	Replayed or masked production-like data	Live RFID/IoT sensor and transaction data
Deployment Trigger	Automatic on every successful build	Automatic after Dev tests pass	Manual approval gate after UAT sign-off
Infrastructure Scale	Minimal (single replica per service)	Near-parity with Production	Full scale, high availability, multi-zone
Access	Developers, CI system	QA, warehouse ops	Restricted; deployment via

Aspect	Development	Staging / UAT	Production
		stakeholders, release manager	pipeline only
Rollback Approach	Redeploy previous build	Redeploy previous release / re-run pipeline	Automated blue-green switch-back

8. Security, Quality Gates, Notifications, and Rollback Mechanisms

8.1 Security Controls

- Secrets (API keys, database credentials, IoT device certificates) stored in a secrets manager (e.g., HashiCorp Vault / cloud KMS), never in source code.
- Container image scanning (Trivy) and dependency vulnerability scanning (Snyk/OWASP) on every build, with critical findings blocking promotion.
- Role-based access control (RBAC) on the CI/CD orchestrator, artifact registry, and Kubernetes clusters, with mandatory code review before merge.
- Encrypted communication (TLS/mTLS) between IoT/RFID gateways, message brokers, and backend services.

8.2 Quality Gates

- SonarQube quality gate enforced at the Code Quality stage: minimum coverage, no new critical/blocker issues, and controlled technical debt ratio.
- Automated regression suite must pass 100% before promotion from Dev to Staging.
- Manual UAT sign-off required before promotion from Staging to Production.

8.3 Notifications

- Pipeline status (build/test/deploy success or failure) posted automatically to a dedicated Slack/Teams channel and via email to the responsible team.
- Production incidents and failed health checks trigger PagerDuty alerts to the on-call engineer.

8.4 Rollback Mechanisms

- Kubernetes rolling-update/blue-green deployments allow instant traffic switch-back to the last known-good version if post-deployment health checks fail.
- All artifacts and Helm chart releases are versioned, enabling one-click redeployment of any previous release from the artifact registry.

- Automated rollback is triggered by monitoring thresholds (e.g., error rate spike, RFID read-failure rate, inventory sync latency) without requiring manual intervention for the initial safety response.

9. Sample Pipeline Configuration (Illustrative)

The excerpt below illustrates, at a conceptual level, how the pipeline stages described in Section 4 would be expressed as pipeline-as-code (e.g., a GitHub Actions / GitLab CI style workflow) for one microservice, such as the RFID/IoT ingestion service. Actual configuration would be split across multiple repositories/services.

```
stages: [build, test, quality_scan, package, deploy_dev, deploy_staging, approve, deploy_prod]

build: docker build -t registry/rfid-ingestion:$CI_COMMIT_SHA .
test: pytest tests/ --cov=app --cov-fail-under=80
quality_scan: sonar-scanner --wait && trivy image rfid-ingestion:$CI_COMMIT_SHA
package: docker push registry/rfid-ingestion:$CI_COMMIT_SHA

deploy_dev: helm upgrade -i rfid-ingestion ./chart -f values-dev.yaml
deploy_staging: helm upgrade -i rfid-ingestion ./chart -f values-staging.yaml
approve_prod: manual_gate(approvers: [release-manager])
deploy_prod: helm upgrade -i rfid-ingestion ./chart -f values-prod.yaml --atomic
  rollback: automatic_on_health_check_failure
```

10. Workflow Explanation: From Code Commit to Deployment

- 1. A developer commits code or opens a pull request; the SCM webhook triggers the CI pipeline.
- 2. The CI orchestrator checks out the code and runs the Build stage, producing a versioned, containerized artifact for the affected microservice.
- 3. Automated unit and integration tests execute; failures stop the pipeline immediately and notify the developer.
- 4. Code quality and security scans run against the build; the pipeline halts if the quality gate is not met.
- 5. On success, the artifact (and, where relevant, the ML model) is pushed to the artifact/container/model registry.
- 6. The pipeline auto-deploys the new artifact to the Dev environment for continued integration testing.
- 7. Automated regression, performance, and security test suites run against Dev; on success the build is promoted to Staging.
- 8. QA and warehouse operations stakeholders perform UAT and canary validation in Staging.
- 9. A release manager reviews Staging results and provides manual approval for Production deployment.
- 10. The pipeline performs a blue-green/rolling deployment to Production, running smoke tests before shifting live traffic.

- 11. Monitoring and logging tools continuously track system health, inventory accuracy, and order-fulfillment metrics.
- 12. Any failure or anomaly triggers automated rollback to the previous stable version, sends alerts to the on-call team, and auto-creates a tracking issue — closing the feedback loop back to the development team.

11. Pipeline Success Metrics (KPIs)

The effectiveness of the CI/CD pipeline itself should be tracked alongside the platform's business metrics, supporting the objective of data-driven warehouse management:

Metric	Target	Why It Matters
Build success rate	> 95%	Indicates code health and stability of the integration branch.
Lead time for changes (commit to Production)	< 1 day for standard changes	Reflects how quickly productivity/accuracy improvements reach warehouse operations.
Deployment frequency	Multiple per week (core services)	Supports continuous delivery of small, low-risk improvements.
Change failure rate	< 10%	Measures how often a Production deployment causes an incident or requires rollback.
Mean time to restore (MTTR)	< 30 minutes	Time to detect and roll back a failed deployment; critical given warehouse operations depend on system uptime.
Test coverage	≥ 80% for core services	Ensures adequate automated protection against inventory-accuracy defects.
Critical vulnerability count at release	0	Enforces the security quality gate before Production promotion.

12. Risks and Mitigation

Risk	Impact	Mitigation
Flaky or slow test suites erode developer trust and slow the pipeline	Delayed releases, developers bypass tests	Quarantine flaky tests, parallelize test execution, monitor test suite duration as a pipeline KPI.

Risk	Impact	Mitigation
AI model degradation after deployment (data drift)	Poor storage/slotting recommendations, reduced productivity	Separate MLOps validation stage with offline evaluation and post-deployment model performance monitoring.
IoT/RFID hardware-software version mismatch across environments	Ingestion failures, inventory sync errors	Environment parity via infrastructure-as-code (Terraform) and versioned device firmware compatibility checks.
Security vulnerability in a third-party dependency	Data breach, compromised warehouse operations	Automated dependency scanning (Snyk/OWASP) on every build with a hard quality gate blocking release.
Failed Production deployment causing warehouse downtime	Halted order fulfillment, financial loss	Blue-green/rolling deployment with automated health-check-based rollback and on-call alerting.
Insufficient staging data realism	Defects surface only in Production	Use replayed/anonymized production traffic and canary releases to a small subset of real Production traffic.

13. Conclusion

The proposed CI/CD pipeline automates the entire path from code commit to production deployment for the Intelligent AI-Based Warehouse Automation and Inventory Tracking Platform. By combining layered automated testing, integrated code quality and security gates, staged Dev → Staging → Production promotion, and real-time monitoring with automated rollback, the pipeline directly supports the project's objectives — improved inventory accuracy, faster order fulfillment, reduced discrepancies, and data-driven, cost-effective warehouse management — while enabling the development team to ship changes to a business-critical system safely and frequently.